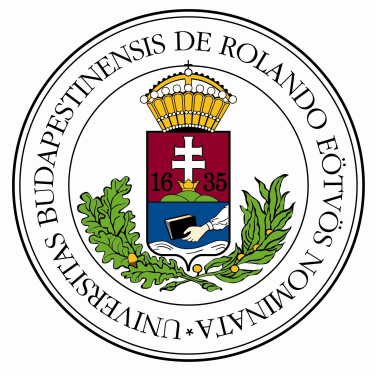
Konkurens programozás

A termelő-fogyasztó feladat



Kozsik Tamás

ELTE Eötvös Loránd Tudományegyetem

Randevú



Termelő-fogyasztó probléma



Termelő-fogyasztó probléma



Termelő-fogyasztó probléma



- Egy folyamat adatokat állít elő egy másik számára
- Nem szinkronizálnak egymással minden adathoz
- A termelő időnként előreszaladhat
- A túl gyors fogyasztó várni kényszerül
- Időben egyenetlen termelés kiegyensúlyozandó

Aszinkronitás mértéke

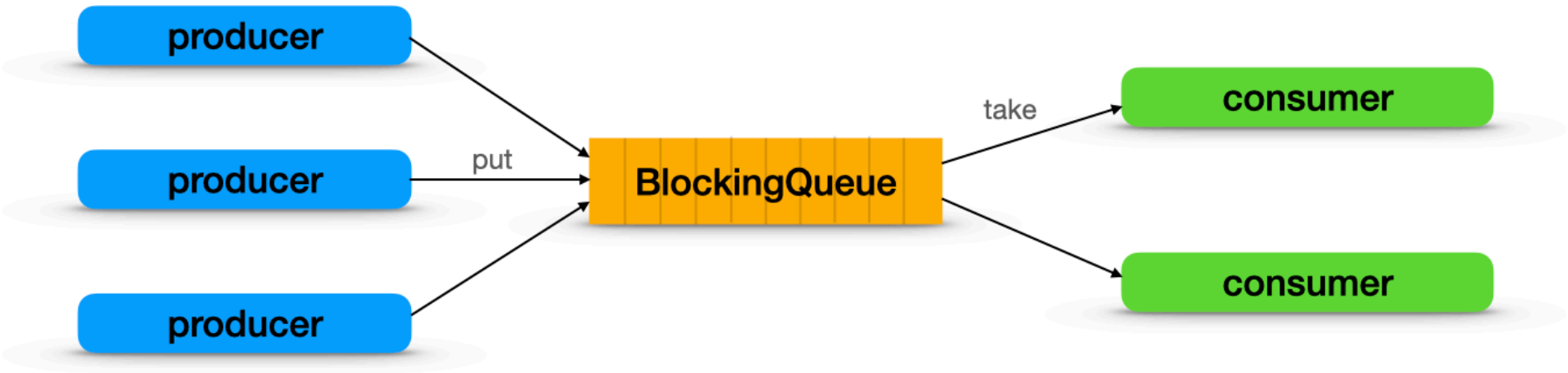
- Randevú (nulla hosszú buffer)
- Egy hosszú buffer
- n hosszú buffer
- Akármekkorára nőni képes buffer



Több termelő, több fogyasztó



A megvalósítás kulcsa: `BlockingQueue`



Többszálú módon használható adatszerkezetek

- `java.util.Vector` és `.Hashtable`, `java.lang.StringBuffer`
- `java.util.Collections.synchronizedList`, `.synchronizedMap`, …
- `java.util.concurrent.ConcurrentMap` és `.ConcurrentHashMap`
- `java.util.concurrent.ConcurrentSkipListMap` és `-Set`
- `java.util.concurrent.CopyOnWriteArrayList` és `-Set`
- `java.util.concurrent.BlockingQueue` és implementációi
- `java.util.concurrent.BlockingDeque` és implementációi
- `java.util.concurrent.ConcurrentLinkedQueue` és `-Deque`



java.util.concurrent.BlockingQueue

- java.util.concurrent.ArrayBlockingQueue
- java.util.concurrent.LinkedBlockingQueue



Megvalósítás: BlockingQueue (1)

```
BlockingQueue<String> buffer = new LinkedBlockingQueue<>();
(new Producer(buffer)).start(); (new Consumer(buffer)).start();
```

```
public class Consumer extends Thread { ... }
```

```
public class Producer extends Thread {
    private final BlockingQueue<String> buffer;
    Producer( BlockingQueue<String> buffer ){ this.buffer = buffer; }
    public void run(){
        while( ... ){
            ...
            buffer.put(...);
            ...
        }
    }
}
```

Megvalósítás: BlockingQueue (2)

```
BlockingQueue<String> buffer = new LinkedBlockingQueue<>();
(new Producer(buffer)).start(); (new Consumer(buffer)).start();
```

```
public class Consumer extends Thread {
    private final BlockingQueue<String> buffer;
    Producer( BlockingQueue<String> buffer ){ this.buffer = buffer; }
    public void run(){
        while( ... ){
            ...
                buffer.take()
            ...
        }
    }
}
```



```
public class Producer extends Thread { ... }
```

Potenciálisan blokkoló művelet

- `BlockingQueue.take()`
- `Thread.join()`
- `Future.get()`
- `ExecutorService.invokeAll(...)`
- `synchronized`
- `DataInputStream.readUTF()`
- `Semaphore.acquire`
- …



“Tetszőlegesen nagyra” nőni képes buffer: rossz ötlet

- Ha a termelő túl gyors, megtelhet a memória
- Jobb lelassítani a termelőt
- Korlátos buffer



“Tetszőlegesen nagyra” nőni képes buffer: rossz ötlet

- Ha a termelő túl gyors, megtelhet a memória
- Jobb lelassítani a termelőt
- Korlátos buffer

```
final static int MAX = 256;
BlockingQueue<String> buffer = new LinkedBlockingQueue<>(MAX);
```



“Tetszőlegesen nagyra” nőni képes buffer: rossz ötlet

- Ha a termelő túl gyors, megtelhet a memória
- Jobb lelassítani a termelőt
- Korlátos buffer

```
final static int MAX = 256;
BlockingQueue<String> buffer = new LinkedBlockingQueue<>(MAX);
```

- Így már a `BlockingQueue.put(...)` is blokkol, ha megtelt a buffer



Konkurens használatra tervezett adatszerkezet műveletei

Throws exception	Special value	Blocks	Times out
add(e)	offer(e)	put(e)	offer(e, time, unit)
remove()	poll()	take()	poll(time, unit)
element()	peek()	-	-



Termelő-fogyasztó probléma leprogramozása

- `java.util.concurrent.BlockingQueue`-val
- `java.io` streamekkel
  - `PipedInputStream` és `PipedOutputStream` (bájtszervezésű)
  - `PipedReader` és `PipedWriter` (karakterszervezésű)

